



**Business Risk
Intelligence Platform**

PLATFORM REFERENCE • V1.0

How the Business Risk Intelligence Platform actually works

A plain-language walkthrough of the architecture, the machine learning models, the data they learn from, and the scoring logic behind two products: a fraud transaction scanner and a business financial health score. Every number in this document is read from the running system, not written down separately from it.

LIVE AT
fdetection.com

BUILT FOR
ReBin Tech

FIRST SHIPPED
16 Sep 2026

CURRENT RELEASE
21 Sep 2026

Three parts, split by what they need to trust

The platform is not one application — it is a public page anyone can use, a full product behind a sign-in, and a small accounts server. Each piece exists because of a specific decision about what has to run where.

PUBLIC · NO ACCOUNT

Free Tools landing page

The site's front door. A basic fraud check and a basic business health check that anyone can try immediately — no sign-up, no data leaving the browser. It exists to let a first-time visitor feel the product work before asking anything of them.

BEHIND SIGN-IN

Customer application

The full platform: CSV upload for both transactions and company financials, portfolio-wide scoring, every chart and explanation, and a saved history of what an account has run. This is where the real work happens.

SERVER-SIDE

Accounts API

A small FastAPI service that does exactly one job: sign-in, sign-up, and sessions. It is deliberately the *only* thing that touches a server. Passwords are hashed server-side, sessions run on short-lived access tokens plus an HttpOnly refresh cookie JavaScript cannot read, and every account check is enforced by the server rather than trusted from the browser.

WHY THIS SPLIT MATTERS

No transaction or financial data ever reaches a server

This is the architecture's central decision, not an afterthought. A transaction CSV or a set of company financials is read directly into the browser, scored by models running in that same browser tab, and never uploaded anywhere. The accounts server never sees it.

Two consequences follow directly: **cardholder and financial data never touch a disk the platform operates**, which keeps the product outside PCI-DSS scope entirely rather than requiring compliance controls around it; and **there is no server-side breach surface for customer data**, because the data is never server-side to begin with.

1 Visitor opens the site

Static files served from one web root — no build step, no server-rendered pages. What loads is exactly what is deployed.

2 Sign-in, if they want the full product

Email and password go to the accounts API over HTTPS. It returns a short-lived access token and sets the refresh cookie. The token lives in memory only — never in localStorage, where a malicious script could read it.

3 Upload a file or type figures in

The file is parsed and scored entirely inside the browser tab. Nothing about the file's contents is sent anywhere.

4 Usage is recorded, not the data

A row is written to the accounts server saying "*this account ran a scan of 900 rows*" — for the account's own history and for the staff console's usage reporting. The 900 rows themselves are never part of that record.

Three models score every case, and vote together

Both the fraud scanner and the health score use the same three-model pattern: two tree-based models that catch non-obvious combinations of risk factors, and a simple linear model kept alongside them as an honest, fully transparent baseline.

MODEL 1 OF 3

Random Forest

Sixty decision trees (fraud) or fifty (business health), each grown on a random resample of the training data and allowed to see only a random subset of the available signals at each split. Averaging many differently-mistaken trees cancels out most of the error any single tree would carry on its own.

Its real strength for this work: a fraud pattern like *"card-not-present, and cross-border, and an amount well above this account's usual spend"* is a combination — none of those three facts alone is meaningful, but together they are a strong signal. A tree represents that exact combination in three nested decisions. A simpler model can only add the three effects up separately, which misses exactly the pattern that matters.

MODEL 2 OF 3

XGBoost (gradient-boosted trees)

Where the forest builds many independent trees, this model builds trees one after another, each one specifically correcting what the trees before it got wrong. 120 rounds, each a shallow 3-level tree.

This is a genuine implementation of the XGBoost training objective — not a simplification of it. Every round computes the true first and second derivatives of the loss function for every row, and a split is only taken if it improves a regularised score:

$$\text{gain} = \frac{1}{2} \left[\frac{G_L^2}{(H_L + \lambda)} + \frac{G_R^2}{(H_R + \lambda)} - \frac{G^2}{(H + \lambda)} \right] - \gamma$$

In practice this model tends to edge out the Random Forest on ranking quality, because later rounds specialise in exactly the borderline cases that a real investigation queue lives in.

MODEL 3 OF 3 · THE HONEST BASELINE

Logistic Regression

A standard, L2-penalised logistic model — the industry-standard interpretable baseline, kept in the results deliberately rather than hidden. It cannot represent the combination-style patterns above; it can only add signals up. It is shown next to the two tree models specifically so the size of that gap is visible and measured, not asserted.

How the three combine into one number

The reported risk score is the average of the Random Forest and XGBoost outputs — the two models that actually drive the decision. The logistic score is shown alongside as the reference point, not blended in. When all three land close together, that agreement itself is meaningful and is surfaced in plain language in the product.

FRAUD · RF TREES

60

FRAUD · BOOST ROUNDS

120

HEALTH · RF TREES

50

HEALTH · BOOST ROUNDS

120

Tree count and boosting rounds are adjustable in the product's Methodology section — changing them retrains the models live and every metric on the page updates, which is one way to verify these are running models rather than fixed numbers.

Synthetic reference data, built to be genuinely hard

No real customer transactions or financials train these models — a documented, reproducible generator builds the reference data fresh every time the platform loads, which is also why no privacy exposure exists in the training data itself.

TRANSACTIONS GENERATED

3,800

fraud training

COMPANIES GENERATED

2,600

health-score training

FRAUD BASE RATE

~3–4%

realistic imbalance

TRAIN / TEST SPLIT

70/30

random, held out

Why "hard" is the design goal

The generator does not scatter risk randomly across features. Fraud is built from four named patterns — card testing, account takeover, "bust-out" fraud, and mule transfers — each one a *combination* of conditions where no single condition alone is suspicious. A card-testing transaction, for example, is deliberately a *small* amount repeated many times through a remote channel — small alone means nothing; small and repeated and remote together is the pattern.

This is what actually separates the tree models from the logistic baseline in the results: the gap in the evaluation numbers is earned by real structure in the data, not asserted by picking numbers that flatter one model.

Every model retrains from scratch, every time

There is no saved model file anywhere in the product. Every time the platform loads, the generator builds a fresh dataset and every model trains against it, live, in the browser. Re-run the same session twice and the models are freshly fit both times — this is what makes the "retrain live and watch the metrics move" verification in the product's Methodology section possible at all.

What the models actually look at

Raw uploaded columns — an amount, a timestamp, a country — are not what the models see. Each is turned into a small set of behavioural signals first, chosen because each one individually has a defensible reason to matter.

Fraud detection — 12 signals per transaction

SIGNAL	WHAT IT MEASURES
Amount (log)	Log-scaled ticket size, so a \$50 and \$50,000 difference doesn't dominate every other signal
Amount vs account norm	This transaction's size divided by that account's own 30-day average
Off-hours timing	Full weight at 23:00–05:00, half weight in the surrounding hours
Card-not-present	Remote channel, no physical card presented
Cross-border	Transaction country differs from the account's home country
Merchant newness	Decays toward zero as the merchant passes one year old
24h velocity	How many transactions this account has made in the last 24 hours
New device	Device not previously seen on this account
Account tenure (log)	Log-scaled days since the account opened
Category risk	The historical fraud rate of this merchant's category
Weekend	Saturday or Sunday
Round-number amount	An exact multiple of 100 — a known card-testing tell

Business health — 10 financial ratios

RATIO	FORMULA
Net margin	Net profit ÷ revenue
Debt-to-assets	Total liabilities ÷ total assets
Current ratio	Current assets ÷ current liabilities
Cash runway	Months of operating expense held in cash
Revenue growth	Year-over-year revenue change
Return on assets	Net profit ÷ total assets
Equity ratio	Net worth ÷ total assets
Interest coverage	Earnings before interest ÷ interest expense
Asset turnover	Revenue ÷ total assets
Working capital position	(Current assets – current liabilities) ÷ assets

Only annual revenue is strictly required on upload. Every other field degrades gracefully to a sector-typical default when missing, and the product tells the user exactly which fields it filled in and why the resulting score carries less confidence.

Two different questions, answered two different ways

"Is this transaction fraud?" and "is this business healthy?" are different enough questions that they use two different scoring approaches, run side by side.

Fraud — a single 0–100% risk score

The ensemble outputs one probability. Five bands turn that into a plain-language read:



The High and Medium cut points are not fixed numbers — they are computed from the held-out test data itself, at the score that best balances catching real fraud against false alarms (and, for the lower Medium band, deliberately biased toward catching more at the cost of a few extra false alarms).

Health — a score out of 100, plus a separate risk read

Two numbers are shown side by side, deliberately not merged into one:

The health score (0–100) is a transparent, auditable sum of four 25-point pillars — profitability, liquidity, solvency, growth — each scored on a published, piecewise-linear formula anyone can check by hand.

The failure probability is the machine-learning ensemble's independent estimate of financial distress within 24 months. It is intentionally kept separate from the pillar score, because the two answer different questions: *how healthy is this business by the numbers* versus *how likely is it to actually fail*.

Investigation queues are ranked by money at risk, not raw score

A 90% risk score on a \$12 transaction is not the first thing worth anyone's time. The fraud queue orders cases by $\text{amount} \times \text{risk probability}$ — expected loss — so the top of the queue is where the real exposure actually sits.

Every score comes with the reasons behind it

A risk score with no explanation is not useful to an investigator or a business owner. Two independent, industry-standard explanation methods run on every case.

METHOD 1

SHAP (Shapley values)

Answers "how much did each individual signal push this specific score up or down?" by testing the prediction with that signal present versus absent, across many random orderings of all the signals, and averaging the effect. It is expensive to compute exactly, so a permutation-sampled approximation runs — 80 permutations per case in the full product, a lighter sample when summarising an entire portfolio at once for speed.

METHOD 2

LIME (local surrogate)

A different question: "if I fit the simplest possible model — a straight line — just in the neighbourhood of this one case, what would it say matters?" It samples 240 nearby points and fits that local model, reporting its own R^2 fit quality alongside the answer, so the reader can see how trustworthy that local approximation actually is.

Why both, not just one

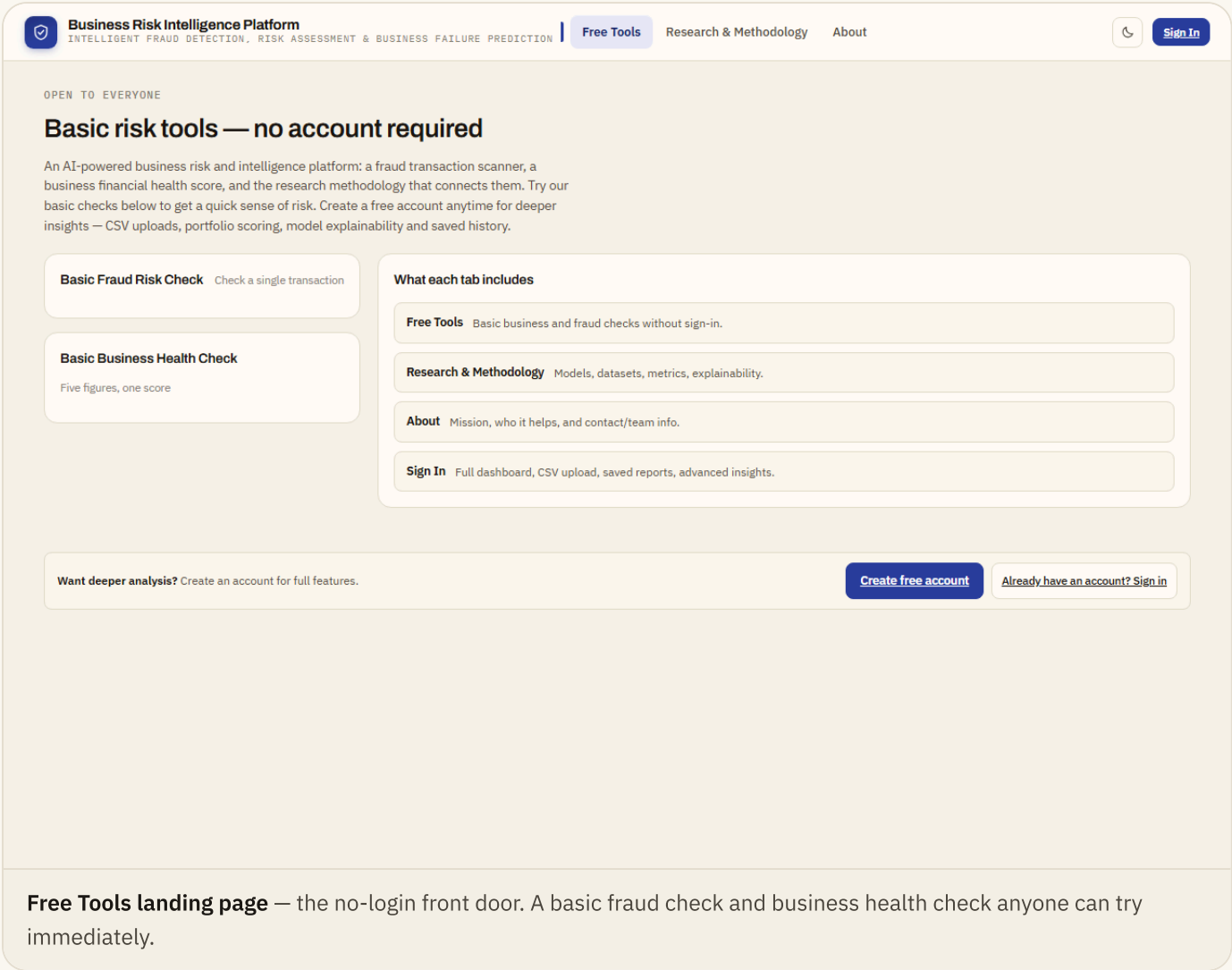
SHAP and LIME can occasionally disagree on a borderline case — that disagreement is itself informative, not a bug to hide. Showing both, with LIME's own fit quality attached, is more honest than picking whichever one tells the cleaner story.

Built for two different readers

The raw SHAP and LIME numbers are real, but they are not written for a business owner — a Shapley value of -0.0412 answers nothing on its own. The product's default view is a plain-language summary — *"the area needing the most attention is liquidity," with one concrete next step* — with the full technical detail (model comparison, the SHAP-weighted driver diagram) tucked behind a collapsed "Advanced details" section for whoever wants to go further.

What it looks like, live

Captured directly from the production site at fdetection.com — not mockups.





Business Risk Intelligence Platform
INTELLIGENT FRAUD DETECTION, RISK ASSESSMENT &
BUSINESS FAILURE PREDICTION

Sign in

Create account

Welcome back

Sign in to scan transactions and score financial health.

Email

you@company.com

Password



Sign in

Sign in — the accounts API is the only thing on the server side; transaction and financial data never pass through it.

Business Risk Intelligence Platform

INTELLIGENT FRAUD DETECTION, RISK ASSESSMENT & BUSINESS FAILURE PREDICTION

Dashboard

Fraud scanner

Health score

Methodology

Activity

TV

Test Verify

DASHBOARD

Your risk desk

What this account has scanned and scored, and where to pick up next.

Welcome back, Test

First sign-in

FILES SCANNED

0

0 transactions

HIGH RISK FOUND

0

nothing scanned yet

EXPOSURE REVIEWED

\$0.00

Sum of expected loss

HEALTH SCORES RUN

3

Last 7 h ago

Scan a transaction file

Upload a CSV and get a queue ordered by expected loss.

Open

Score a business

Type the figures in, or score a whole portfolio from a file.

Open

See how it decides

The models, the held-out metrics and the run log behind every score.

Open

Built as a self-contained browser application: no server, no database, and no external machine-learning library. The tree ensembles, the logistic baseline, the transformer encoder and the SHAP and LIME explainers are implemented in assets/ml.js and execute locally every time the platform scores. Reference datasets are synthetic and generated in-page from documented processes, so nothing you load leaves the browser.

Dashboard — an account's own risk desk: what it has scanned and scored, and where to pick up next.

SECTION 01 - TRANSACTION SCANNER

Find the fraud buried in the file

Upload a transaction CSV and the platform maps the columns, derives twelve behavioural features and scores every row through a Random Forest and gradient-boosted ensemble.

The queue that comes back is ordered by expected loss, and each case names the features that put it there.

Transaction file

Drop a transaction file

Excel or CSV — or click to choose

Loaded: transactions_sample.csv — 900 rows, 16 columns

Column mapping

13 of 16 detected

Transaction amount	REQ	Txn Amount (USD)
Timestamp		Posted On
Transaction ID		— not mapped —
Account / customer ID		— not mapped —
Channel		Entry Mode
Merchant name		Payee Name
Merchant category		— not mapped —
Merchant age (days)		Payee Age (d)
Transaction country		Txn Country
Account home country		Billing Country

Analyse transactions

Operating thresholds

High risk at or above

0.171

TRANSACTIONS

900

\$51.0K total value

CRITICAL RISK

4

0.4% of file

HIGH RISK

11

1.2% of file

MEDIUM RISK

84

9.3% of file

AVERAGE RISK SCORE

3.5%

Base rate 4.3%

VALUE AT RISK

\$3.6K

Expected loss, amount × risk

FLAGGED EXPOSURE

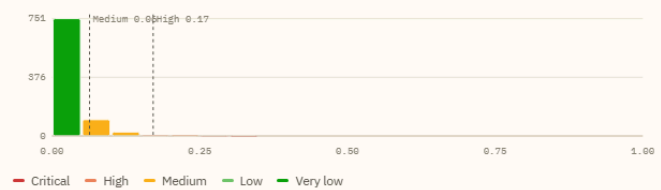
\$21.7K

Face value of flagged rows

1 field imputed. Merchant category were not in the file. The scorer substituted per-account medians where possible and a neutral default otherwise, so those features carry less signal than they would on a complete feed.

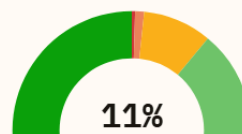
Risk distribution

Scored in 25 ms



Risk mix

Share of file

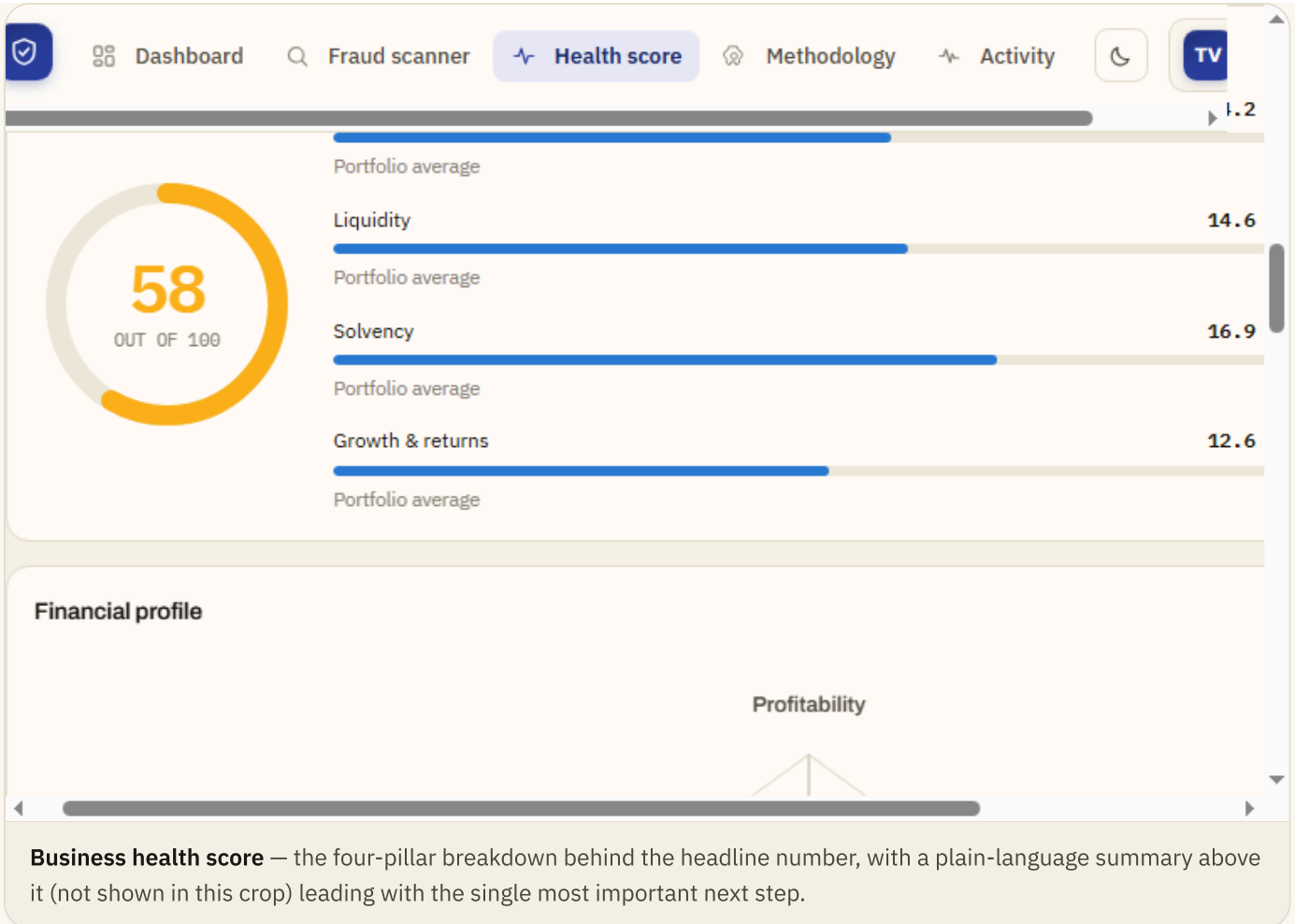


What is driving the risk

Averaged across all scored rows

Amount vs account norm	18%
Amount (log)	16%
Account tenure (log)	13%

Fraud scanner, mid-scan — 900 real sample transactions scored in 25ms: risk distribution, a risk-mix donut, and the signals driving the score, averaged across the file.



From first commit to current release

Every entry below is a real, shipped change — read from the project's own commit log.

16 Sep 2026

First commit — the platform's foundation

Core scanner, health score and methodology sections; the three-model ensemble; sign-in and the staff admin console added the same day.

16 Sep 2026

Real backend, HttpOnly refresh cookies, Excel upload support

18 Sep 2026

Demo-mode auth removed; upload safety hardening

Every account check now runs against the real server — no client-only fallback left anywhere in the product.

19 Sep 2026

Public Free Tools landing page shipped, and root-domain launch

The no-login page moved to the site root; clean URLs (/login, /admin) replaced raw filenames.

21 Sep 2026

Fraud scanner expanded to 5 risk bands, with a risk-mix chart and a low-data warning

21 Sep 2026

Four new visual explanations added to the health score

A financial-profile radar, a model-comparison chart, a balance-sheet bridge, and a pillar-to-driver diagram — for both single companies and uploaded portfolios.

21 Sep 2026

Plain-language summary added; technical detail collapsed by default

Every score now leads with one verdict sentence and a single next step, in plain language — the current release.

Full commit-by-commit history is available in the project's Git log on request.

Who built it, and where the code lives

Repository

The full source — frontend, backend, and this documentation's source facts — lives in a single Git repository, deployed directly to the production server on every release.

FRONTEND

Vanilla JS

no framework, zero build step

ML IMPLEMENTATION

Hand-written

no ML library dependency

BACKEND

FastAPI

Python, accounts only

HOSTING

Self-managed

nginx + Let's Encrypt TLS

Who is who in the product

RT

ReBin Tech

Operator — runs the platform, holds the staff accounts, owns the staff console

C

Customers

The businesses that sign up and use the scanner and health score — never see the staff console

Deployment

Static frontend assets served directly from the web root; the accounts API runs as an independent, isolated system service so a frontend deploy can never affect account sessions in progress, and vice versa.

Where the platform stands today

The platform shipped its first public release five days before this document. Usage evidence below reflects that: an early-stage, functionally complete product, not yet a measured rollout.

What exists today

A working accounts system with real sign-up, sign-in, and per-account usage history; a staff console reporting sign-ins, files scanned, and health scores run across every account; and a public, no-login entry point specifically built to lower the barrier to a first try.

What adoption evidence looks like from here

The staff console (reachable at `/admin` with a staff account) already tracks the numbers that will populate this section as real usage accumulates: accounts created, sign-ins, files scanned, and health scores run, each timestamped per account. This section is intended to be revisited and filled in with real figures once the platform has been live long enough to report them honestly.

This section is deliberately left thin rather than backfilled with placeholder numbers — an adoption claim is only worth making once there is real usage behind it.

Prepared from the running system at `fdetection.com` — every figure in this document is read from the product's own source and its live commit history, not maintained separately from it.